

Instructions to Run the project

Prerequisites (environment)

The deployment of the project relies on

- **docker**,
- **docker compose**, and
- **make**.

These dependencies must be installed in order to run the project.

The program has been tested on Linux, "*Ubuntu 24.04*".

Instruction to run the code and tests

The construction and execution of all docker containers, as well as the code, is done through the make utility, to simplify the overall execution of the project.

All the following make commands can take the argument `TARGET_SERVICES` from the command line, to execute only the needed services:

- e.g. **make build TARGET_SERVICES="auth order"** will perform the build of only auth and order services.

The value that can be passed to such argument are:

- for target run (code section): *auth*, *product*, *cart*, *order*, *orchestrator*;
- for target test (test section): *auth_test*, *product_test*, *cart_test*, *order_test*.

It is particularly useful while executing tests, to see the results of tests for each single service, but not really interesting for the project execution.

If the `TARGET_SERVICES` argument is not used, all the services for the current target will be started: do not use this argument while running the project because all sw components are needed.

Code Section

Compile and run the code

To compile and run the project, execute the following commands in order from the root folder of the project (where the Makefile is located):

1. **make build**
2. **make up** (or **make up_bg**, to execute the code in the background: logs will not be displayed)

Once the containers are running, open a browser on the machine where the project is executed and go to

- <http://localhost:3000> (or use a different port if you have configured the variable `WEB_EXPOSE_PORT` in the *deployment/.env* file with a different value)

to access the e-commerce web interface.

See the “Use Cases” section in the report to understand how to use the website.

Note: Starting all components may take several seconds. If some required components have not yet started, you may receive an Internal Server Error. Wait a few seconds and the system will be fully operational.

Stop the execution

- If you used **make up**, to stop all containers press **Ctrl+C** in the same terminal.
- If you used **make up_bg**, to stop all containers, execute **make stop**.

Clean Up

Docker containers are configured with volumes to persist data across executions. To remove all Docker volumes and networks created for the e-commerce system, use the command **make down**.

After running **make down**, you must execute **make build** again before using **make up** to restart the project.

Test section

Compile and run all tests

To build containers, compile .proto files and run all tests, execute the following command from the root folder of the project (where the Makefile is located):

- **make run_test**

The results of all tests will be displayed in the terminal.

Compile and run tests for one or more services

To build containers, compile .proto files and run all tests for one or more services, execute the following command from the root folder of the project (where the Makefile is located):

- **make run_test TARGET_SERVICES="**<service_name_1> <service_name_2> .. <service_name_n>**"**
 - if more services are between “ ”, they must be separated by a single space.

where each **<service_name_i>** value is in the following list:

- **auth_test**, to execute test for the auth service;
- **product_test**, to execute test for the product service;
- **cart_test**, to execute test for the cart service;
- **order_test**, to execute test for the order service.

Stop the execution

To stop the execution of all docker containers used for running tests, use the command **make stop_test**.

Clean up

To remove all Docker volumes and networks created for the e-commerce system, use the command **make down_test**.

Clean up the system after executions

After the execution of the program and / or tests, you can remove all docker containers, volumes and network by using the command **make purge** (which calls stop, stop_test, down and down_test).